

Sistemas Operativos

1º Parcial 2C2025 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría – Single Choice

- 1- E/S por DMA
- 2- Ninguna de las anteriores
- 3- No permiten paralelismo entre hilos pares
- 4- Cambio de modo y de contexto
- 5- VRR
- 6- La sección crítica es corta y hay paralelismo
- 7- Espera limitada

Teoría – Desarrollar

1.

Planificador a largo plazo:

- Transiciones de estado que administra:
 - new → ready (decide si un nuevo proceso entra al sistema).
 - También controla exit cuando un proceso finaliza.
- Puede basarse en la mezcla de procesos CPU-bound vs I/O-bound para optimizar el uso de CPU y los dispositivos de I/O.
- Sí, modifica directamente el grado de multiprogramación

Planificador a mediano plazo:

- Transiciones de estado que administra:
 - ready/blocked → suspended ready/blocked (swap out, suspende un proceso).
 - suspended ready/blocked → ready/blocked (swap in, lo devuelve a memoria).
- Puede basarse en la disponibilidad de RAM y la prioridad de los procesos para liberar memoria cuando sea necesario o reactivar algún proceso para mejorar la mezcla de procesos CPU-bound e I/O-bound
- Modifica el grado de multiprogramación al suspender o reactivar procesos.

Planificador a corto plazo:

- Transiciones de estado que administra:
 - ready → running (elige qué proceso ejecuta la CPU).
 - running → ready (cuando desaloja un proceso).
- Basado en algoritmos de planificación de CPU: FIFO, Round Robin, SJF, prioridades, etc.
- No modifica el grado de multiprogramación, solo decide el orden de ejecución.

2. Una condición de carrera ocurre cuando dos o más procesos/hilos acceden de forma concurrente a un recurso compartido (variable, archivo, estructura de datos) y al menos uno de ellos lo modifica, de manera tal que el resultado final depende del orden de ejecución. Esto genera resultados no determinísticos e inconsistentes.

<pre>int contador = 0; //variable compartida mutex = 1</pre>	
Hilo 1	Hilo 2
<pre>while(true) { wait(mutex) / habilitarInterrupciones() contador ++; signal(mutex) / deshabilitarInterrupciones() }</pre>	<pre>while(true) { wait(mutex) / habilitarInterrupciones() contador ++; signal(mutex) / deshabilitarInterrupciones() }</pre>

Desventajas de deshabilitar interrupciones:

- No escalable en multiprocesadores: habría que enviar la orden de deshabilitar interrupciones a todos los núcleos, con un gran costo.
- Los procesos de usuario no están habilitados a realizar esta operación.

3.

La prevención consiste en diseñar el sistema de tal manera que al menos una de las 4 condiciones necesarias para el deadlock no pueda cumplirse: Exclusión mutua, Retención y espera, No expropiación (sin desalojo) y Espera circular.

El objetivo es que nunca pueda formarse el ciclo de espera que produce el deadlock.

Ejemplos de implementación:

- Ejemplo 1 – Eliminar “retención y espera”. Un proceso debe pedir todos los recursos que va a necesitar de una vez, si no los consigue todos, no se le da ninguno.
- Ejemplo 2 – Eliminar “espera circular”. Se define un orden global de petición de recursos, cada recurso tiene un número de prioridad, y los procesos solo pueden pedirlos en orden creciente.

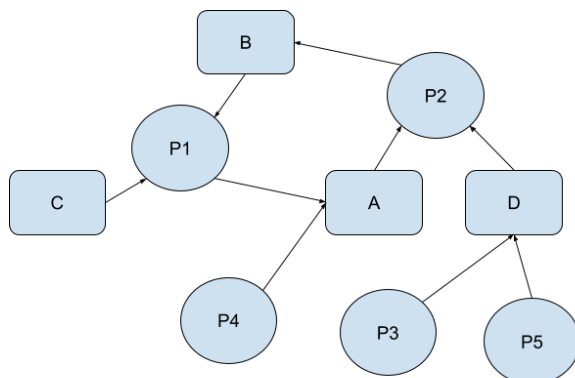
Evasión es más flexible, porque no restringe la forma de pedir recursos a priori, aunque exige conocer las necesidades máximas de cada proceso. Luego, ante cada petición, decide si realiza o no la asignación del recurso. Solo se concede la petición si el sistema queda en un estado seguro.

4.

mutexPendientes = mutexValidados = 1
 tomarCafe[N] = {0,0,0,...}, capacidadPreparador = 50
 preparador = 0, semValidados = 0

Cliente (N instancias)	Procesador (1 instancia)	Robot Barista (3 instancias)
<pre> while(true){ pedido = generarPedido() wait(capacidadPreparador) wait(mutexPendientes) agregar(pedido, pendientes) signal(mutexPendientes) signal(preparador) wait(tomarCafe[getId()]) tomarCafe() } </pre>	<pre> while(true){ wait(preparador) wait(mutexPendientes) pedido = retirar(pendientes) signal(mutexPendientes) signal(capacidadPreparador) validar(pedido) wait(mutexValidados) agregar(pedido, validados) signal(mutexValidados) signal(semValidados) } </pre>	<pre> while(true){ wait(semValidados) wait(mutexValidados) pedido = retirar(validados) signal(mutexValidados) cafe = hacer_cafe(pedido) servir(cafe) signal(tomarCafe[cafe.idCliente]) } </pre>

5.



Los procesos en deadlock son P1 y P2.

P3, P4 y P5 solamente sufren starvation ya que están bloqueados indefinidamente pero no retienen recursos.